DRep Expiry & Dormant Epoch Tracking

DRep Expiry & Dormant Epoch Tracking

Overview

In the Cardano Conway era, Delegated Representatives (DReps) have an **expiry epoch** that determines whether they are "active" for governance voting. If a DRep's expiry has passed, they are excluded from the ratification tally — their delegated stake does not count.

The expiry extends whenever a DRep interacts with the chain (registers, votes, or updates their metadata). Additionally, during **dormant periods** (epochs with no active governance proposals), all DRep expiries are effectively extended so that DReps are not punished for inactivity when there is nothing to vote on.

This document covers how DRep expiry is tracked across four implementations: **Haskell** (canonical reference), **Amaru** (Rust), **Yano** (Java node), and **Yaci Store** (Java indexer).

Key Concepts

DRep Activity Parameter

The protocol parameter `drepActivity` (typically 20 on mainnet) defines how many epochs a DRep stays active after their last interaction. If a DRep registers at epoch 520 with `drepActivity = 20`, their initial expiry is epoch 540.

Dormant Epochs

An epoch is **dormant** when there are no active governance proposals remaining after ratification processing at its boundary. During dormant epochs, DReps cannot vote (nothing to vote on), so their expiry should not tick down.

Conway Phases

Conway governance has two phases, determined by the protocol version:

Phase	Protocol Version	Description
Bootstrap (V9)	Major version 9	Initial Conway era. DRep voting thresholds for most actions are

Phase	Protocol Version	Description
		effectively 0 (SPOs and Constitutional Committee decide). DRep registration and voting exist but DRep votes don't drive ratification for most proposal types.
Post-Bootstrap (V10+)	Major version 10+	Full Conway governance. DRep votes actively determine ratification outcomes. DRep expiry handling becomes more precise — the dormant counter is subtracted at registration/vote/update time.

The transition from V9 to V10 happens via a hard fork initiation proposal that is itself ratified under V9 rules.

---

1. Initial Expiry at Registration

When a DRep registers, they receive an initial expiry epoch.

V9 (Bootstrap Phase)

All implementations agree:

```
initialExpiry = registrationEpoch + drepActivity
```

Example: DRep registers at epoch 520, `drepActivity = 20` -> expiry = 540.

The dormant counter is **not** subtracted. This is because in V9, the dormant counter handling at epoch boundaries (via flush or on-the-fly addition) will extend the expiry implicitly.

V10+ (Post-Bootstrap Phase)

```
initialExpiry = registrationEpoch + drepActivity − numDormantEpochs
```

The subtraction of `numDormantEpochs` compensates for the fact that at ratification time, `numDormantEpochs` will be added back. Without this subtraction, a DRep registering during a dormant period would get a double benefit.

Example: DRep registers at epoch 560 when `numDormantEpochs = 5`, `drepActivity = 20`: - Stored expiry = 560 + 20 - 5 = 575 - At ratification, effective expiry = 575 + 5 = 580 - Net effect: 560 + 20 = 580 (same as intended)

Implementation	V9 Formula	V10 Formula
Haskell	`epoch + drepActivity`	`epoch + drepActivity` (V10 subtraction not in local checkout — may be in newer branch)
Amaru	`epoch + drepActivity`	`epoch + drepActivity - consecutive_dormant_epochs`
Yano	`epoch + drepActivity`	`epoch + drepActivity - numDormantEpochs`
Yaci Store	Computed post-hoc from history	Computed post-hoc from history

---

2. Dormant Epoch Counter

Each implementation tracks whether epochs are dormant and maintains a counter.

When is an epoch dormant?

At each epoch boundary (N -> N+1), after ratification and expiry processing: - Count the **remaining active proposals** (proposals that were not ratified or expired at this boundary) - If remaining count = 0 -> the epoch is **dormant** - If remaining count > 0 -> the epoch is **not dormant**

Important: The Conway first epoch (epoch 507 on mainnet) is always dormant because no proposals can exist before governance bootstraps.

Counter behavior by implementation

Implementation	Storage	Increment	Reset/Flush
Haskell	`vsNumDormantEpochs` (single integer in VState)	+1 at EPOCH rule when previous epoch had empty proposal snapshot	Reset to 0 when a transaction containing a new proposal triggers the CERTS flush
Amaru	`consecutive_dormant_epochs` (persisted integer)	+1 at epoch boundary when no active proposals remain	**Never resets**. The counter monotonically increases. V10 registration subtracts it, and reads add it back.
Yano	`numDormantEpochs` (RocksDB key 0x6E)	+1 at epoch boundary when `!epochHadActiveProposals`	Reset to 0 during flush at epoch boundary (when proposals exist and counter > 0)
Yaci Store	`gov_epoch_activity.dormant_epoch_count` (DB table)	+1 when current epoch is dormant	Resets to 0 on any non-dormant epoch

Example: Mainnet epochs 507-511

Epoch	Proposals remaining?	Haskell counter	Yano counter	Amaru counter
507 (Conway start)	0 (no proposals yet)	0 -> 1	0 -> 1	0 -> 1

Epoch	Proposals remaining?	Haskell counter	Yano counter	Amaru counter
508	Yes (first proposals submitted)	1 -> 0 (flush)	1 -> 0 (flush)	1 (stays, never resets)
509	Yes	0	0	1
510	Yes	0	0	1
511	Yes	0	0	1

---

3. Stored Expiry Modification at Epoch Boundaries

This is where the implementations diverge most significantly in their internal mechanics, while producing equivalent results.

Haskell: Flush-on-proposal (in CERTS rule)

Haskell does **not** flush at the epoch boundary. Instead, the flush happens during **transaction processing** in the CERTS rule:

```
-- When a transaction contains governance proposals AND numDormantEpochs > 0:
if hasProposals && isNumDormantEpochsNonZero
  then
    -- Add numDormantEpochs to EVERY DRep's stored drepExpiry
    certState & vsDRepsL %~ (<&> (drepExpiryL %~ (+ numDormantEpochs)))
    -- Reset counter to 0
    & vsNumDormantEpochsL .~ 0
```

After the flush, `drepExpiry` contains the fully materialized value. The ratification check is simply `currentEpoch > drepExpiry`.

Key detail: The flush iterates ALL registered DReps (the entire `vsDReps` map), regardless of whether they are expired or active. There is no non-revival guard — an expired DRep gets bumped but may still be expired after the bump.

Yano: Flush-at-boundary (in GovernanceEpochProcessor)

Yano flushes at the **epoch boundary** rather than during transaction processing:

```
if (epochHadActiveProposals && numDormant > 0) {
    // FLUSH: bump every registered DRep's stored expiry
    for (DRepStateRecord state : allDRepStates) {
        // Skip deregistered tombstone records
        if (isDeregistered(state)) continue;
```

```
        int bumped = state.expiryEpoch() + numDormant;
        // Non-revival guard: don't write bumped value if still expired
        int newExpiry = (bumped < newEpoch) ? state.expiryEpoch() : bumped;
        boolean active = newExpiry >= newEpoch;
        store(newExpiry, active);
    }
    resetCounter(0);

} else if (!epochHadActiveProposals) {
    incrementCounter(numDormant + 1);
}
```

Yano's non-revival guard: If `bumped < newEpoch`, the DRep's stored expiry is left unchanged. This prevents writing a higher (but still expired) value. This is an optimization — the DRep stays expired either way, but Yano avoids unnecessary writes.

Timing equivalence: Yano flushes at the epoch boundary before ratification. Haskell flushes when the first proposal-bearing transaction arrives (which is within the same epoch boundary processing). Since both flush before ratification runs, the effective expiry during tally is identical.

Amaru: Never flush (compute-on-the-fly)

Amaru never modifies stored `valid_until` at epoch boundaries. Instead, it adds the counter at read time:

```
// When building governance summary for ratification:
DRepState {
    valid_until: Some(stored_valid_until + consecutive_dormant_epochs),
    ...
}
```

Since `consecutive_dormant_epochs` monotonically increases and V10 registration subtracts it, the math works out: - Stored value = `regEpoch + activity − D_at_registration` - Effective = stored + D_current = `regEpoch + activity + (D_current − D_at_registration)` - `(D_current − D_at_registration)` = dormant epochs accumulated since registration

Yaci Store: Full recomputation each epoch

Yaci Store doesn't maintain live state. At each epoch boundary, it recomputes every DRep's expiry from the full history (registration epoch, last vote/update epoch, dormant epoch set, V9 bonus). This is expensive but trivially correct and rollback-safe.

---

4. Effective Expiry for Ratification

At ratification time, each implementation determines which DReps are "active" (their delegated stake counts in the tally).

Implementation	Active check	Where
Haskell	`currentEpoch > drepExpiry` -> expired (excluded)	`Ratify.hs: dRepAcceptedRatio`
Amaru	`valid_until > Some(epoch)` -> active	`dreps.rs: is_active()`
Yano	`effectiveExpiry >= newEpoch` -> active, where `effectiveExpiry = storedExpiry + numDormantEpochs`	`GovernanceEpochProcessor: buildActiveDRepKeys()`
Yaci Store	Checks computed `active_until` column	`DRepExpiryService`

Boundary semantics

The exact boundary condition matters: - **Haskell**: `currentEpoch > drepExpiry` means a DRep with `drepExpiry = 540` is active at epoch 540, expired at epoch 541. - **Yano**: `effectiveExpiry >= newEpoch` means a DRep with `effectiveExpiry = 540` is active at epoch 540, expired at epoch 541. - **Amaru**: `valid_until > Some(epoch)` means a DRep with `valid_until = 540` is active at epoch 539, expired at epoch 540.

Amaru's boundary is off-by-one relative to Haskell/Yano: it uses strict `>` while the others use `>=` (Haskell) / `>=` (Yano). This is compensated by Amaru computing `valid_until` differently (likely 1 higher in practice).

---

5. Expiry Refresh on Vote and Update

When a DRep votes or updates their metadata, their expiry is refreshed.

V9 (Bootstrap Phase)

Implementation	On Vote	On Update
Haskell	`drepExpiry = currentEpoch + drepActivity`	`drepExpiry = currentEpoch + drepActivity`
Yano	Only `lastInteractionEpoch` updated (expiry	Only `lastInteractionEpoch` updated

Implementation	On Vote	On Update
	recalculated at boundary)	
Amaru	No explicit expiry refresh visible	No explicit expiry refresh visible
Yaci Store	Records interaction; expiry recomputed at next boundary	Records interaction; expiry recomputed at next boundary

In V9, Haskell directly sets the stored expiry on vote/update. Yano tracks the interaction epoch and lets the epoch boundary recalculate. The result is equivalent because the V9 expiry formula is `max(regEpoch, lastInteractionEpoch) + drepActivity + dormantCount`.

V10+ (Post-Bootstrap Phase)

Implementation	On Vote	On Update
Haskell	`drepExpiry = currentEpoch + drepActivity`	`drepExpiry = currentEpoch + drepActivity`
Yano	`storedExpiry = currentEpoch + drepActivity − numDormantEpochs`	`storedExpiry = currentEpoch + drepActivity − numDormantEpochs`
Amaru	No explicit refresh visible	No explicit refresh visible

In V10, Yano subtracts the pending dormant counter (same as registration). Haskell does NOT subtract because in Haskell, the flush has already materialized all dormant epochs into the stored value — there is no pending counter to subtract.

---

6. The V9 Bonus (Bootstrap Compatibility)

DReps that registered during the initial dormant bootstrapping period of Conway (before any proposals existed) have a special case. In Haskell, when the first proposal arrives and triggers a flush, ALL DReps get `numDormantEpochs` added to their stored expiry. This includes DReps registered during the dormant period — they effectively get a “bonus” extension.

For example, on mainnet: - Conway starts at epoch 507 (dormant — no proposals) - First proposal submitted in epoch 508 block -> triggers flush with counter=1 - A DRep registered at epoch 507 with `drepActivity=20`: - Initial stored expiry: 507 + 20 = 527 - After flush: 527 + 1 = 528 - Net effect: as if registered at epoch 508

Yaci Store and the older Yano code explicitly computed this as a "V9 bonus" to replicate the Haskell behavior. In the current Yano counter-based approach, the bonus emerges naturally from the flush — no special handling needed.

---

7. DBSync `active_until` Convention

DBSync's `drep_distr.active_until` column represents the **effective expiry at snapshot time**, which is `drepExpiry + vsNumDormantEpochs` (Haskell's on-the-fly computation). It is NOT the raw stored `drepExpiry`.

This means: - `active_until` changes across epochs even if the DRep does nothing, because `vsNumDormantEpochs` accumulates - It jumps when dormant epochs are flushed (the flush materializes into `drepExpiry`, then `vsNumDormantEpochs` resets) - Direct comparison with Yano's `stored_expiry` is not meaningful — they use different internal representations

For correctness verification, compare: - **DRep counts** (should match exactly) - **Delegated amounts** (should match within staking reward timing tolerance) - **Ratification outcomes** (the definitive test — same proposals ratified/rejected at same boundaries)

---

8. Correctness Assessment

Is Yano's implementation correct according to Haskell?

Yes, with high confidence. The mathematical equivalence holds:

1. **Registration formula**: V9 `epoch + drepActivity` matches Haskell. V10 `epoch + drepActivity − numDormant` compensates correctly for the pending counter.
2. **Flush semantics**: Yano flushes at epoch boundary; Haskell flushes on first proposal-bearing transaction. Both happen before ratification, so the effective expiry during tally is identical.
3. **Counter tracking**: Yano increments on dormant epochs, resets on flush — same as Haskell's `vsNumDormantEpochs`.
4. **Active check**: `effectiveExpiry >= newEpoch` produces the same result as Haskell's `currentEpoch > drepExpiry` (both: DRep with expiry=540 is active at 540, expired at 541).
5. **Flush scope**: Yano iterates ALL registered DReps (`getAllDRepStates()`), skipping only deregistered tombstones. This matches Haskell which iterates the full `vsDReps` map (deregistered DReps are

removed from Haskell's map entirely).

6. **Non-revival guard**: Yano has an explicit guard (`bumped < newEpoch ? oldExpiry : bumped`). Haskell does not need one because the ratification check (`currentEpoch > drepExpiry`) handles it implicitly — a DRep that is still expired after the bump is simply excluded from the tally.

What matters: ratification outcome

The ultimate test is not whether intermediate stored values match (they can't — different internal representations), but whether the **ratification tally at each epoch boundary produces the same set of active DReps with the same effective expiry**. This is what determines whether proposals get ratified or rejected.

Verification at epoch 575 (the critical boundary for the -12T treasury fix) will confirm this.

Appendix: File References

Haskell (cardano-ledger)

- `libs/cardano-ledger-core/src/Cardano/Ledger/DRepDistr.hs` — `DRepState` record
- `libs/cardano-ledger-core/src/Cardano/Ledger/CertState.hs` — `VState` , `vsNumDormantEpochs`
- `eras/conway/impl/src/Cardano/Ledger/Conway/Rules/GovCert.hs` — registration/update rules
- `eras/conway/impl/src/Cardano/Ledger/Conway/Rules/Certs.hs` — flush on proposal + vote expiry refresh
- `eras/conway/impl/src/Cardano/Ledger/Conway/Rules/Epoch.hs` — `updateNumDormantEpochs`
- `eras/conway/impl/src/Cardano/Ledger/Conway/Rules/Ratify.hs` — `dRepAcceptedRatio`

Amaru (Rust)

- `crates/amaru-ledger/src/summary/governance.rs` — `GovernanceSummary` , effective expiry computation
- `crates/amaru-ledger/src/rules/transaction/phase_one/certificates.rs` — V9/V10 registration
- `crates/amaru-ledger/src/state.rs` — dormant counter increment
- `crates/amaru-ledger/src/governance/ratification/dreps.rs` — `is_active()` check

Yano (Java)

- `ledger-state/.../governance/epoch/GovernanceEpochProcessor.java` — `updateDRepExpiry()` , `buildActiveDRepKeys()`
- `ledger-state/.../governance/GovernanceBlockProcessor.java` — `processDRepRegistration()` , vote/update refresh
- `ledger-state/.../governance/GovernanceStateStore.java` — `getNumDormantEpochs()` , `storeNumDormantEpochs()`
- `ledger-state/.../governance/model/DRepStateRecord.java` — stored record

Yaci Store (Java)

- `aggregates/governance-aggr/.../util/DRepExpiryUtil.java` — full expiry calculation
- `aggregates/governance-aggr/.../service/DRepExpiryService.java` — epoch boundary computation
- `aggregates/governance-aggr/.../processor/GovEpochActivityProcessor.java` — dormant tracking